6th IEOM India International Conference • Bangalore, India • 5–7 November 2026

Design and Implementation of Two Virtual Reality Competency Trainers for Manual Assembly and Hand-Tool Selection on a Common Extended Reality Platform

Prof. V Vasu[1] • **Binet N B**[1] • **Mula Hruthik**[2]

[1] Department of Mechanical Engineering, National Institute of Technology Warangal, Telangana, India
[2] Independent AR/VR developer, Warangal, Telangana, India



Interaction is now cheap — competency trainers are not

Manual assembly remains skill dependent

It absorbs part-to-part variation, handles compliant components and supports low-volume, high-mix work without retooling — so quality is bound to operator competence.

VR is an established training medium

No material use, no hazard exposure, unlimited repetition, direct instrumentation of trainee behaviour (Boud et al. 1999; Carlson et al. 2015).

Standalone headsets removed the cost barrier

Inside-out tracking and bare-hand recognition: immersive training no longer needs a specialist facility.

Commercial interaction framework supplies (configured)

Grasping | Ray pointing | UI pointing | Hand-pose authoring | Locomotion

→ *an object that can be picked up*

A competency trainer also needs (must be written)

- A representation of the task
- Validation of what the trainee did against it
- Guidance that advances with progress
- Recording of performance, where scored

The path from a working grasp system to a working trainer is not well documented.

Prior work shows VR training works — not what it takes to build

Study	Scope	What it establishes
Daling & Schlittmeier (2024)	Scoping review: AR/VR/MR for manual assembly	Effects on objective and subjective measures; protocol structure explains much outcome variation
Di Pasquale et al. (2024)	Systematic review: VR assembly & disassembly	Structured assessment remains uncommon
Yang et al. (2023)	VR in manufacturing education	Effectiveness reviewed across disciplines and environments
Wang et al. (2022)	VR engine-assembly trainer (Unity, Quest 2)	Competency in fewer attempts than video demonstration
Alfaro-Viquez et al. (2025)	Meta Quest 3 welding trainer — closest system	User evaluation: reduced execution time, fewer positioning errors
Lin et al. (2024); Eswaran et al. (2024)	AR instruction authoring	Task specification derived as data from the product/CAD model
This paper	**Two Quest trainers, one platform**	**What is built above the framework, and its cost (5,186 lines); decisions compared**

GAP

None of these reports what must be built above a commercial interaction framework to obtain a competency trainer — nor how much effort that layer represents.

Objectives and contributions

Objectives

1. Describe two VR competency trainers for different industrial tasks, built on one XR platform
2. Identify what a commercial interaction framework supplies and what must be developed — and quantify that effort
3. Compare task specification, validation and guidance strategies; identify the common design principle
4. Report verification methods usable when interaction cannot be exercised outside the headset

Contributions

Boundary made explicit

Framework vs. custom application layer, quantified: 5,186 lines of code across 21 source files

Design decisions isolated

Two independent builds on one platform as parallel cases → one shared principle: derive the task specification from the scene

Device-free verification

Programmatic scene checks and seeded, headless simulation of session logic

Scope: an account of implementation — not a new interaction technique, not a training algorithm, and not a claim of instructional effectiveness.

Two trainers analysed as parallel cases on one platform

1. **Parallel cases**
   Two trainers built independently by the authors for different tasks; never merged

2. **Characterise each system**
   Task model · validation rule · guidance strategy · assessment behaviour

3. **Locate the framework boundary**
   Census of interaction components; custom code counted in lines and files

4. **Verify without a device**
   Programmatic scene checks; seeded headless session simulation (System B)

Shared platform (Table 1)

Property	System A	System B
Engine	Unity 6000.0.43f1	Unity 6000.3.10f1
Render pipeline	URP 17.0.4	URP 17.3
Interaction framework	Meta Interaction SDK 201.0.0	Meta Interaction SDK 203.0.0
XR backend	OpenXR 1.14.3	OpenXR 1.16.1
Target hardware	Meta Quest 3, standalone	Meta Quest 3 and 3S, standalone
Task model	Authored sequence, 26 steps	Seeded generation from scene scan

Why comparable: same engine family, framework, device and build method → differences reflect design decisions, not platform.

Qualification: different authors, so only structural decisions traceable to each task's requirements are compared.

System A — guided assembly of a 26-component electric motor

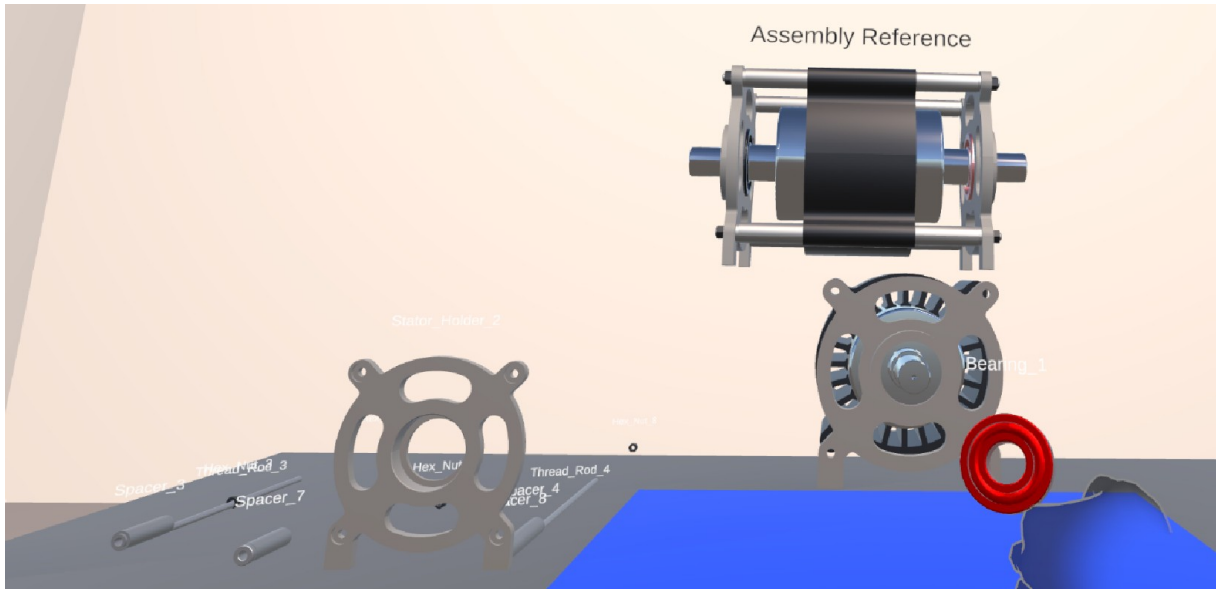


Fig. 1 Workspace with the highlighted active component and the 3D target-state display

26
components — only 2 unique

26 × 2
JSON steps: assembly + disassembly

4
action classes: place · insert · fasten · remove

2,422
lines / 10 files (state machine 772)

Placement validation

Scene start: record the world pose of every component while the motor is still assembled → placement targets

↓

Parts scattered; trainee grasps and releases one

↓

Check: is it the component this step requires — and within 0.15 m of its stored pose? (tolerance configurable)

Both hold: snaps to exact pose and locks

Either fails: rejected, part stays free

Guidance — 3 concurrent channels: workspace highlight · continuous 3D target-state display · progress panel (informational)

Matters later: the 8 “fasten” steps resolve through the placement path, and no assessment data is recorded.

System B — hand-tool selection from a rack of 88 industrial tools

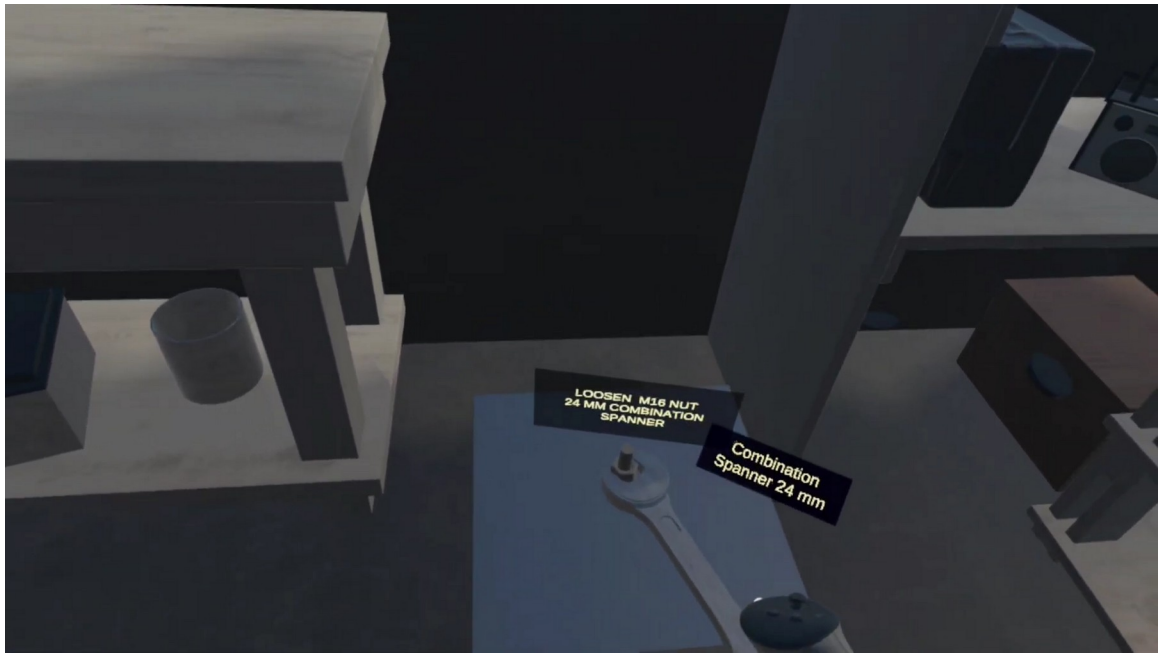


Fig. 2 Job label and a 24 mm combination spanner seated on the fastener by its working end

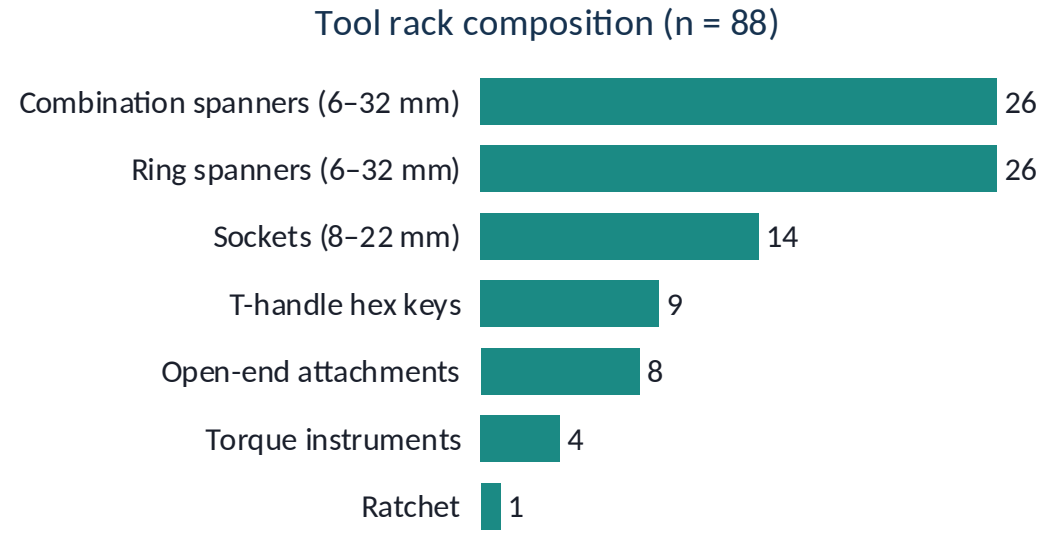
Observe
90 s · no task · unscored

→

Guided
required tool is named

→

Assessed
tool not named · scored



Deliberate gaps (no 31 mm spanner, no 20 mm socket) → legal sizes come from a scan of the rack at session start

Score per job (100 points)



Accepted only if type, size (within tolerance) and, where applicable, set torque all match. Rejected → fastener does not turn; tool retained for the debrief. 2,764 lines / 11 files.

Interaction was configured — the training layer was written

Training layer — written by the authors

- Task representation
- Object identity, type and size
- Validation of trainee action
- Fastener engagement and driving
- Guidance that advances with progress
- Performance recording and scoring

Framework boundary

Interaction framework — obtained by configuration (Meta Interaction SDK)

- Grasp by tracked hand and controller
- Ray selection and UI pointing
- Authored hand poses
- Constrained single-axis rotation
- Locomotion and comfort vignette

5,186

lines of custom application code across 21 source files

A **2,422 lines · 10 files**

B **2,764 lines · 11 files**

Framework side, System B:

183 ray interactables · 94 grabbables · 4 rotation transformers · 5 pointable canvases — with no interaction code written.

Adapted from Fig. 3 and Table 3 of the paper

RESULT 2 · DESIGN DECISIONS

Same problems, different decisions — one shared principle

Design problem	A · Guided assembly	B · Tool selection
Task specification	Authored sequence in an external JSON file; 26 steps, 4 action classes	Generated at session start from a rack scan, under coverage, interleaving and exclusion constraints
Source of ground truth	Component poses captured from the product model at scene start	Tool types and sizes enumerated from the tagged rack
Validation of trainee action	Correct component for the step, placed within 0.15 m of the captured pose	Correct type, size within tolerance and, where applicable, set torque within tolerance
Guidance strategy	Three concurrent channels: highlight, 3D target state, progress panel	Staged: 90 s observation → jobs naming the tool → jobs that do not
Failure behaviour	Placement rejected; component remains free	Fastener does not turn; rejected tool retained for the debrief

SHARED PRINCIPLE · REACHED INDEPENDENTLY

Derive the task specification from the scene — don't author it in code

- **A** Poses captured from the product model → a revised model needs no re-targeting
- **B** Rack scanned each session → tools that are not on the wall are never requested

Otherwise a stale specification is scored as a trainee error (cf. Lin et al. 2024)

RESULT 3 · SOFTWARE VERIFICATION

Verification without a headset: seeded, headless sweeps

Tracked hands are unavailable outside the headset, so System B's session logic is driven headlessly; because generation is seeded, every generated job can be inspected without a device.

685
torque jobs generated (200 seeds × 8 jobs per session) and checked

0
defects: missing instrument, unreachable target, or undialable target

0.060
N·m per degree, identical on all 4 torque instruments

0
sizes shared between training and assessment sets

15 / 15
generated torque labels fit the plate

Property (Table 5)	Method	Result
Rotation progress retention	Numeric simulation	360° forward, then 180° reverse → retains 360°
Torque accumulation past wrap	12 stops to 500° handle travel	Reads 54.99 N·m; monotonic past the wrap point
Session determinism	Seed 777 vs. seed 12345	1 of 10 slots identical

Shows

System B behaves as specified; System A verified by enumeration against its component manifest

Does not show

Learning outcome (no user data), or that no unknown class of defect exists

Each system implements what the other omits

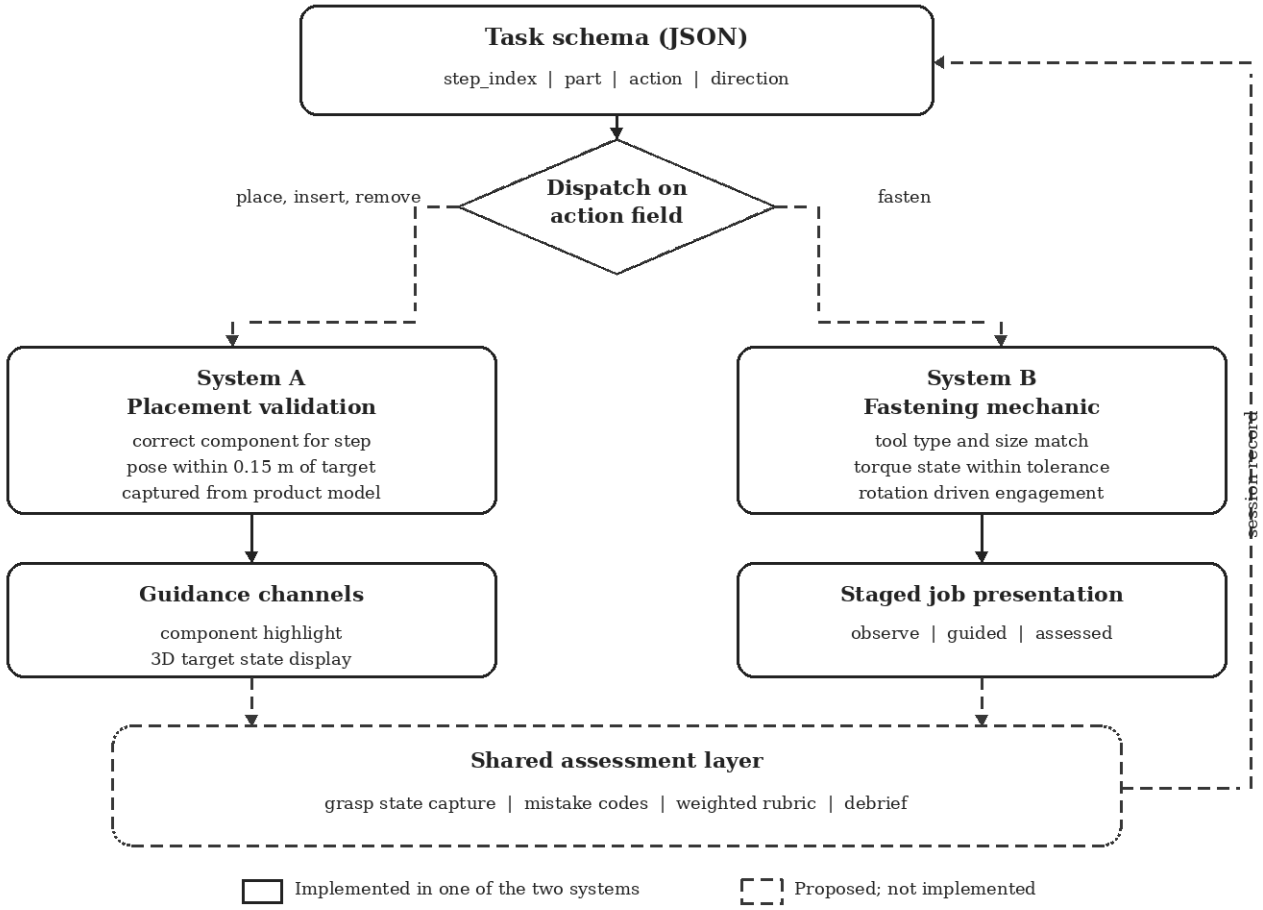


Fig. 4 Solid = implemented in one of the two systems; dashed = proposed, not implemented

PROPOSAL — NOT IMPLEMENTED

A Names “fasten” (8 steps) in its schema, but resolves it as placement

B Simulates fastening (tool, size, torque), but each fastener has no assembly context

1 · Dispatch on the action field

place / insert / remove → A’s validation; fasten → B’s mechanic. One task definition = a complete work instruction.

2 · Shared assessment layer

Extend B’s grasp-state capture to A’s placement errors, which A currently discards.

Both already broadcast state through event interfaces — no change to A’s assembly logic needed.

What this work contributes — and where it stops

Contributions

- The framework boundary, made explicit and quantified: 5,186 lines, 21 files
- A side-by-side comparison of design decisions taken independently on one platform
- Verification methods for when interaction cannot be exercised off-device

Practical implications

- Plan for the training layer: task, validation, guidance and assessment are written, not configured
- Derive the task specification from the scene, so product revisions and rack changes do not become false trainee errors
- Seeded generation lets every session be checked without a headset

Limitations

- No user study: no claim on learning outcome, skill acquisition or transfer
- Evidence from two systems, two developers, one platform
- The integration is a proposal only; System A records no assessment data

Conclusion and future work

All four objectives addressed

1. Both trainers described by task model, validation, guidance and assessment (Sec. 3.2–3.3)
2. Framework supplies interaction; the training layer took 5,186 lines in 21 files (Sec. 3.4)
3. Shared principle: derive the task specification from the scene (Sec. 5.1)
4. Device-free verification: 685 generated jobs checked headlessly (Sec. 3.6, 5.4)

Next steps (from the stated proposals and limitations)

- Implement action-field dispatch: one work instruction spanning placement and fastening
- Extend grasp-state assessment to System A's placement errors
- User study to measure learning outcome and transfer

Interaction can now be configured. The competency trainer — task, validation, guidance, assessment — still has to be written.

Thank you — Questions?

Acknowledgement

The authors thank the ASEAN collaborative project between the Department of Mechanical Engineering, National Institute of Technology Warangal, and Universiti Malaysia Pahang Al-Sultan Abdullah, under which part of this work was undertaken.

Contact

vasu@nitw.ac.in · bn25mem3r03@student.nitw.ac.in · mulahruthik2000@gmail.com



How the reported numbers were obtained

System B verification gate (Sec. 3.6)

1. **Structural assertions**
   Programmatic checks over the scene graph

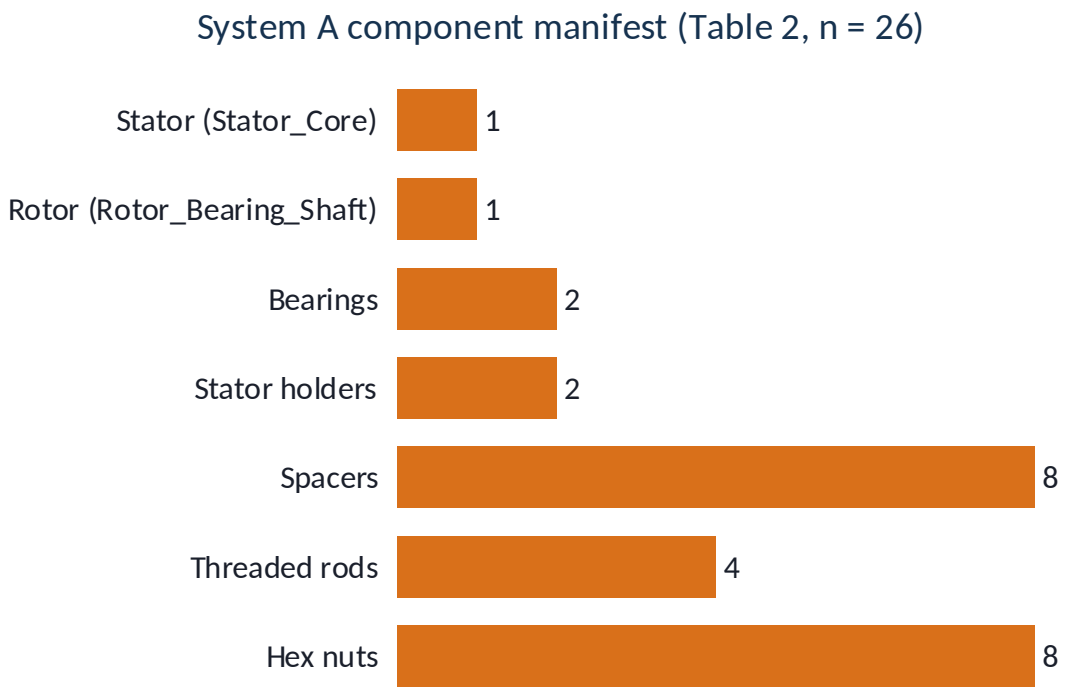
2. **Clean play-mode console**
   Any error or warning fails the gate (hard gate)

3. **Headless session layer**
   Drives session logic through reflection: generates and steps through complete sessions so labels, task sequences, scoring and text layout are verified in bulk

Data provenance (Sec. 4)

- System A counts are exact: enumerated from the component verification list and cross-checked against the task configuration file
- Rack composition enumerated from the tagged tool set in the deployed scene
- Interaction component counts: census over every object in the scene, including inactive objects
- System B scene: 458 mesh instances, 2,106,767 triangles, 63 materials (1,172,734 triangles in the 88 tools)
- No human participant data collected; all results are properties of the software

Component manifest and task-generation rules



JSON step entry: step index · part name · action (place / insert / fasten / remove) · insertion direction. 24 of 26 parts belong to groups of geometrically identical members.

System B job generation (seeded)

- Coverage: every playable family appears at least once
- Interleaving: identical neighbouring families separated after shuffling
- No recurrence of a size within a fixed number of jobs
- Assessment list uses the training list as an exclusion set
- Same seed → same session, so an instructor can reissue it

Assessment capture

- Grasp state of every tool polled for a rising edge → one event per pickup
- Weights 60 / 20 / 10 / 10: efficiency smallest, since rewarding speed on a torque operation would reinforce the wrong behaviour